CSS4003 — DevOps 2

Lecture 10: Docker in Production

Nauruzbayeva Farikha · Lesson 10





Recap — Lesson 9 (Cloud Storage & Docker Fundamentals)

- Image vs container? (an image is a read-only template; a container is a running instance of it with a thin writable layer)
- Why copy `requirements.txt` before `app.py`? (so the `pip install` layer stays cached when only the code changes)
- Where did the notes survive `docker rm`? (in a named volume, which outlives any container)
- Why were the keys in `.env` and not in the image? (anything in an image is readable by everyone who can pull it)

Today’s agenda

- ☐ Registries: Docker Hub and GitHub Container Registry
- ☐ Tags that mean something: semver, git SHA, digests, multi-arch
- ☐ A production-ready image: small, non-root, with a healthcheck
- ☐ Security gate: scanning with Trivy, SBOMs
- ☐ Running on a Linux server with Compose: restarts, limits, logs, secrets
- ☐ Releasing v2 and rolling back to v1
- ☐ Why orchestration — a first look at Kubernetes and k3s
- ☐ Live demo → straight into today’s practice

The scenario

Last week the Flask app finally ran the same everywhere — on **your** laptop. Now the team wants it online. Someone suggests: "just SSH into the server, `git clone`, `pip install` and run it."

Ship the exact image you tested to a server, keep it running, update it safely, and roll back in seconds.

Block 1

Registries

Build once, run anywhere

- The server should **never build** the app. It should **pull** the exact image you tested.
- A **registry** is where images live between "built" and "running".
- The same image (same **digest**) runs on your laptop, in CI and on the server — so "it worked in testing" actually means something.

```
laptop / CI ──docker push──▶ registry ──docker pull──▶ server(s)
```

Registry, repository, tag

```
ghcr.io / farikha / lab10-app : v1
└─────┘   └─────┘   └───────┘   └┘
registry  owner     repository  tag
```

- **Registry** — the server that stores images (`docker.io` , `ghcr.io` , or your own).
- **Repository** — one app; it holds all its versions.
- **Tag** — a human-readable name for one version.
- No registry part (`nginx:1.27`) means **Docker Hub** by default.

Docker Hub vs GitHub Container Registry

	Docker Hub	GHCR
Image name	`docker.io/<user>/app:1.0`	`ghcr.io/<user>/app:1.0`
Login	`docker login` + access token	`docker login ghcr.io` + GitHub token
Lives next to	Docker's public images	Your GitHub repo and Actions
Visibility	Public / private repos	Public / private packages
Free tier	Limited private repos, pull rate limits	Free for public packages

Both speak the same registry protocol: `docker push` and `docker pull` work the same everywhere. Next week GitHub Actions pushes to GHCR for us.

Logging in to GHCR

```
read -s CR_PAT                        # paste a GitHub token, never echo it
echo "$CR_PAT" | docker login ghcr.io -u <github-user> --password-stdin
# Login Succeeded
```

- The token is a **Personal access token** with `write:packages` (to push) — or only `read:packages` on a server that just pulls.
- `--password-stdin` keeps the token out of shell history and `ps`.
- Docker stores it in `~/.docker/config.json` — that file is a secret too.
- Prefer one token per machine, revoke it when the machine goes away.

Tag and push

```
docker build -t lab10-app:v1 .
docker tag lab10-app:v1 ghcr.io/farikha/lab10-app:v1
docker push ghcr.io/farikha/lab10-app:v1
docker pull ghcr.io/farikha/lab10-app:v1      # on any machine
```

- `docker tag` doesn't copy anything — it adds a second **name** for the same image.
- The registry part of the name tells `docker push` **where** to upload.
- Only layers the registry doesn't have yet are uploaded: the second push is much faster.

Your own registry

```
docker run -d --name registry --restart unless-stopped -p 5001:5000 registry:2.8.3
docker tag lab10-app:v1 localhost:5001/lab10-app:v1
docker push localhost:5001/lab10-app:v1
curl -s localhost:5001/v2/lab10-app/tags/list
```

- `registry:2` is the open-source registry — useful in labs, air-gapped networks and on-prem.
- No auth and no TLS out of the box: fine on `localhost`, never on the internet.
- Port 5001 on macOS: port 5000 is taken by AirPlay.

Block 2

Tags, digests, architectures

Tags that mean something

Tag	Example	Use
Semantic version	`v1.4.2`	Releases people talk about
Git commit SHA	`sha-3f9c2e1`	Exactly which code is inside
Branch / env	`main`, `staging`	Convenience — moves every build
`latest`	`latest`	Default when no tag is given — **moves**

Good practice: push **both** `v1.4.2` and `sha-3f9c2e1` for every release. Deploy by version, debug by SHA.

Why not `latest`?

- `latest` is just a tag — nothing guarantees it's the newest or tested.
- Two servers pulling `latest` an hour apart can run **different** code.
- A rollback to "the previous `latest`" is impossible: it was overwritten.
- In production, every deploy names an **explicit version**.

"It worked yesterday" + `:latest` = you don't know what ran yesterday.

Tag vs digest

```
docker image inspect --format '{{index .RepoDigests 0}}' ghcr.io/farikha/lab10-app:v1
# ghcr.io/farikha/lab10-app@sha256:a9b241a9…
docker pull ghcr.io/farikha/lab10-app@sha256:a9b241a9…
```

- A **tag** is a label that can be moved to another image.
- A **digest** (`sha256:…`) is computed from the content: same digest = byte-for-byte the same image, forever.
- Pull by digest when you must be sure — e.g. in a security-reviewed deploy.
- `docker inspect <container>` shows which image ID a container runs.

The `arm64` vs `amd64` trap

- Apple Silicon Macs build `arm64` images by default.
- Most servers and cloud VMs are `amd64` (`x86_64`).
- Run an `arm64` -only image there → `exec format error` , container exits.
- Check: `docker image inspect --format '{{.Architecture}}' <image>`

Multi-arch images with buildx

```
docker buildx create --use --name multi
docker buildx build --platform linux/amd64,linux/arm64 \
  -t ghcr.io/farikha/lab10-app:v1 --push .
docker buildx imagetools inspect ghcr.io/farikha/lab10-app:v1
```

- One tag, a **manifest list** with one image per platform: each machine pulls the one that matches its CPU.
- The default `docker` builder can’t push multi-platform images — create a `docker-container` builder first (line 1).
- Building the other architecture uses emulation: slower, but it works.

Block 3

A production-ready image

Small images: multi-stage builds

```
FROM python:3.12 AS build
COPY requirements.txt .
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
COPY --from=build /install /usr/local
COPY app.py /app/
CMD ["python", "/app/app.py"]
```

- Compilers and build tools stay in the first stage; only results are copied into the final image.
- Smaller image = faster pulls and **fewer packages to have CVEs**.
- Bases from big to small: `python:3.12` → `-slim` → `-alpine` / distroless.

Don’t run as root

```
RUN useradd --create-home appuser && mkdir -p /data && chown appuser /data
USER appuser
```

- By default the process in a container is **root** (uid 0).
- If the app is exploited, root inside the container is one bug away from root on the host.
- `USER` makes everything after it — including `CMD` — run as that user.
- Check: `docker exec <container> whoami` → `appuser`.

HEALTHCHECK

```
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/healthz')"
```

- Docker runs the command every 10 s; 3 failures in a row → `unhealthy`.
- `docker ps` shows `(healthy)` / `(unhealthy)` next to `Up …`.
- "Running" only means the process exists. **Healthy** means it answers.
- No `curl` in the slim image? Use the language you already have.

`.dockerignore` and secrets at build time

```
.venv/
__pycache__/
.env
secrets/
*.pem
```

- Everything not ignored is sent to the builder — and can end up in a layer with `COPY . .`.
- A secret deleted in a later layer is **still** in the earlier one: `docker history` and `docker save` reveal it.
- Rule: **no secret ever goes into an image** — not in `COPY`, not in `ENV`, not in `ARG`.

Block 4

Security gate: Trivy

Scanning images with Trivy

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:0.69.3 image --severity HIGH,CRITICAL --exit-code 1 lab10-app:v1
echo $?          # 1 = findings at that severity → the gate fails
```

- Trivy matches the packages in the image against known vulnerabilities (CVEs): OS packages and Python/Node/… libraries.
- `--exit-code 1` turns the scan into a **gate**: a script or CI step fails when it finds something.
- Most findings come from the **base image**.

Reading the results

- **Fixed Version** present → you can fix it: rebuild on a fresh base (`docker build --pull`) or upgrade the package.
- No fix yet → `--ignore-unfixed` hides it; record it and re-check later.
- Accepted risks go into a `.trivyignore` file **with a reason**, never silently.
- Scan again after every rebuild: new CVEs are published every day.

```
RUN apt-get update && apt-get -y upgrade && rm -rf /var/lib/apt/lists/*
```

SBOM: what’s inside

```
trivy image --format cyclonedx lab10-app:v1 > sbom.json
```

- **SBOM** — Software Bill of Materials: every package and version in the image, in a standard format (CycloneDX, SPDX).
- When the next big CVE appears, you search SBOMs instead of guessing which images are affected.
- Increasingly required by customers and regulators.

Quick check

Your server runs `app:latest`. After a bad release you want the previous version back. What's the problem, and what should the deploy have used?

`latest` was overwritten, so "the previous one" has no name. Deploy explicit versions (`v1`, `v2` or a SHA tag) — then rollback is just running the previous tag.

Block 5

Running on a Linux server

Docker on the server

```
curl -fsSL https://get.docker.com | sudo sh     # official convenience script
sudo usermod -aG docker $USER                    # log out and back in
docker version && docker compose version
```

- Works on any Linux VM: a university server, any cloud provider, a VM on your laptop.
- Membership in the `docker` group is effectively **root** on that machine — give it only to admins.
- Open only what you need in the firewall: SSH from your IP, the app port.

Restart policies

Policy	Restarts when…
`no` (default)	never
`on-failure`	the process exits with an error
`always`	it exits for any reason, and when Docker starts
`unless-stopped`	like `always`, except after you `docker stop` it

```
docker run -d --restart unless-stopped -p 8080:8000 ghcr.io/farikha/lab10-app:v1
```

The app crashes at 3 a.m. → Docker starts it again; the server reboots → the container comes back.

Healthcheck vs restart policy

- **Restart policy** reacts when the process **exits**.
- **Healthcheck** detects a process that is alive but **not answering** (deadlock, stuck DB connection) and marks it `unhealthy`.
- Plain Docker does **not** restart an `unhealthy` container on its own — orchestrators (Kubernetes, Swarm) do.
- Compose can wait for health: `depends_on: { db: { condition: service_healthy } }`

Compose on the server

```yaml
services:
  app:
    image: ${IMAGE}:${TAG}          # from .env — no build: on servers
    restart: unless-stopped
    ports: ["8080:8000"]
    volumes: [notes:/data]
    mem_limit: 256m                 # resource limits
    cpus: 0.5
    secrets: [api_token]            # mounted at /run/secrets/api_token
    logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }
secrets:
  api_token: { file: ./secrets/api_token.txt }
volumes:
  notes:
```

Resource limits

- Without limits, one leaking container can eat all RAM and take every other service on the host down with it.
- `mem_limit: 256m` — above that the kernel kills the container (OOM), and the restart policy brings it back.
- `cpus: 0.5` — at most half a CPU core, so one busy container can't starve the rest.
- Check: `docker stats` and `docker inspect --format '{{.HostConfig.Memory}}' <c>`.

Logs

```
docker compose logs -f --tail 50 app
```

- Write logs to **stdout/stderr** — Docker collects them; no log files in the container.
- The default `json-file` driver **never rotates**: logs can fill the disk. `max-size: "10m"` + `max-file: "3"` caps them at ~30 MB.
- With many servers you ship logs to one place (Loki, Elasticsearch, a cloud log service) and search them there — coming with monitoring.

Configuration vs secrets

	Example	Where it lives
Config	`TAG=v1`, `PORT=8080`, log level	`.env`, environment variables
Secret	DB password, API token	a **file**: Compose/Docker secrets, mounted at runtime

- Environment variables leak easily: `docker inspect`, crash dumps, child processes. Files under `/run/secrets` don't show up there.
- Many apps support `*_FILE` variables: `DB_PASSWORD_FILE=/run/secrets/db_password`.
- Secret files stay out of git (`.gitignore`) and out of the image (`.dockerignore`). In a team they come from a secret manager.

Block 6

Updates and rollbacks

Releasing v2

```
docker buildx build --platform linux/amd64,linux/arm64 -t $IMAGE:v2 --push .
ssh server "sed -i 's/^TAG=.*/TAG=v2/' lab10/.env"
ssh server "cd lab10 && docker compose pull && docker compose up -d"
curl -s http://server:8080/version
```

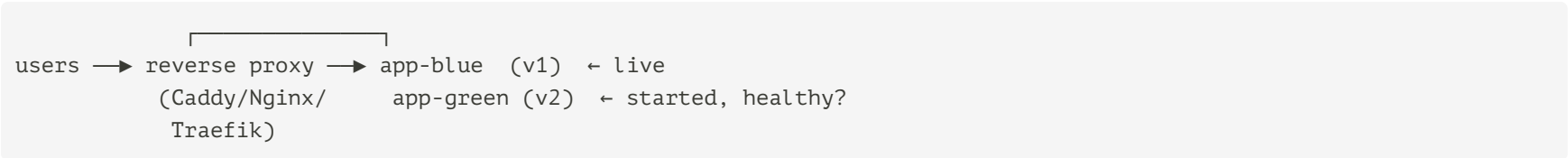
- Compose recreates only the containers whose image or config changed.
- Volumes stay: data from v1 is still there for v2.
- A few seconds of downtime while the container is replaced.

Rollback in seconds

```
ssh server "sed -i 's/^TAG=.*/TAG=v1/' lab10/.env && cd lab10 && docker compose up -d"
```

- v1 is still in the server's local image cache: no download needed.
- Rollback is only this easy because every release has its **own tag**.
- Keep the last few versions on the server; `docker image prune` the rest — carefully.

Less downtime: blue/green



- Start v2 **next to** v1, wait until it's healthy.
- Switch the proxy to green, then stop blue.
- Something wrong? Switch back — blue is still running.
- Doing this by hand on many servers gets painful… which is the next block.

Quick check

The container shows `Up 2 hours (unhealthy)`. Why didn't `restart: unless-stopped` restart it?

The restart policy only reacts when the process exits. An unhealthy process is still running. Plain Docker reports it; an orchestrator (or your own monitoring) has to act on it.

Block 7

Why orchestration

One server is not enough

- **Scheduling** — which of 20 machines has room for this container?
- **Self-healing** — restart unhealthy containers, move them off a dead machine.
- **Scaling** — 3 copies now, 10 during registration week.
- **Rolling updates** — replace copies one by one, roll back automatically.
- **Service discovery** — find "the API" without hard-coding IPs.

An orchestrator does all of this. The standard one is Kubernetes.

Kubernetes in one slide

Object	What it is
Pod	One or more containers scheduled together; the smallest unit
Deployment	"Keep N pods of this image running" + rolling updates
Service	A stable name and IP in front of changing pods
Ingress	HTTP routing from outside into Services
ConfigMap / Secret	Configuration and secrets for pods
HPA	Scales a Deployment by CPU or other metrics

You describe the **desired state**; Kubernetes keeps reconciling the real state towards it.

k3s: Kubernetes for small machines

- **k3s** — a certified Kubernetes distribution in a single small binary.
- Runs on a 1–2 GB VM, a Raspberry Pi, or in Docker (**k3d**).
- Install on a Linux server: `curl -sfL https://get.k3s.io | sh -`
- Comes with `kubectl`, an ingress controller (Traefik) and local storage.
- Your **final project** runs on exactly this kind of cluster.

The same app, on Kubernetes

```
kubectl create deployment lab10 --image=ghcr.io/farikha/lab10-app:v1 --replicas=2
kubectl expose deployment lab10 --port=80 --target-port=8000
kubectl scale deployment lab10 --replicas=3
kubectl set image deployment/lab10 lab10-app=ghcr.io/farikha/lab10-app:v2
kubectl rollout status deployment/lab10
kubectl rollout undo deployment/lab10
```

Compare with Compose: no SSH, no `sed` on `.env`. Kubernetes replaces pods one by one and remembers the history for `rollout undo`.

Block 8

Checklist and troubleshooting

Production checklist

- ☐ Explicit version tags (and a SHA tag) — never deploy `latest`
- ☐ Multi-arch, or built for the server's CPU
- ☐ Small base, multi-stage, non-root `USER`, `HEALTHCHECK`
- ☐ Trivy gate passing, accepted findings documented
- ☐ Restart policy, memory/CPU limits, rotated logs
- ☐ Secrets as files, never in the image, `.env` or git
- ☐ A tested rollback

When it doesn’t work

Symptom	Likely cause
`exec format error`	Image built for the wrong CPU (arm64 vs amd64)
`denied` / `unauthorized` on push/pull	Not logged in, wrong token scope, upper-case name
`(unhealthy)`	Healthcheck URL/port wrong, app not on `0.0.0.0`
Container restarts in a loop	App crashes at start: `docker logs` ; OOM: check `mem_limit`
Disk full	Unrotated logs or old images: `docker system df`
Secret missing	Wrong path, or file not readable by the container user

Block 9

Today’s practice

Today’s practice — ship it

1. Harden last week’s image: non-root, `HEALTHCHECK`
2. Push **v1** to a registry: GHCR (Track A) or a local registry (Track B)
3. Run it on a "server" with Compose: restart, limits, log rotation, secret from a file
4. Release **v2**, roll back to **v1**
5. Crash it and watch Docker restart it
6. Trivy gate: fail → fix → pass

```
docker compose ps
# app  Up 30 seconds (healthy)
curl -s localhost:8080/version
# {"version":"v2", …}
sed -i 's/TAG=v2/TAG=v1/' .env
docker compose up -d
# {"version":"v1", …}
```

Stretch: the same app on a local Kubernetes cluster — `scale`, `set image`, `rollout undo`.

By the end of this lesson, you should be able to

- ☐ Push an image to a registry with meaningful tags and explain tag vs digest
- ☐ Build an image that runs on both `amd64` and `arm64`
- ☐ Make an image smaller, non-root, healthchecked and scanned
- ☐ Run it on a Linux server with Compose: restarts, limits, logs, secrets
- ☐ Release a new version and roll back in seconds
- ☐ Explain what an orchestrator adds and name the core Kubernetes objects

Before next lecture

- ☐ Finish today’s practice if you didn’t wrap it up in session
- ☐ Create a GitHub repository for your final project from the template
- ☐ Make sure you can push to GHCR (token with `write:packages`)

Next week the pipeline does the build, scan and push for you.

Next lecture

CI/CD with GitHub Actions — build, test, scan and push the image automatically on every commit.